PROGRESS REPORT · V1.0 · 21 OCTOBER 2026

GraSyn: what we have built so far

A collaboration workspace for small software teams, where tasks, decisions, conversations, and now GitHub code live in one connected place. This report covers everything finished in Phase 1 (the core workspace), Phase 2 (live teamwork: live updates, presence, project discussions, and meeting records), and Phase 3 (task keys and GitHub: pushes and pull requests linked to tasks), on both the backend and the frontend.

Backend: NestJS · Prisma · PostgreSQL · Socket.IO

Frontend: React · Vite · Tailwind · TanStack Query · GSAP

Private repository: github.com/ahmei1/Grasyn

66
API endpoints, plus a live socket connection

24
database tables, plus 6 status and role lists

16
app screens, plus the landing story page

107
automated tests passing (102 unit and 5 on a real database)

Where the project stands

Phases 1, 2, and 3 are complete. A project now holds the work, the conversation about it, and the code that implements it: tasks with short keys, a live discussion, meeting records whose action items become real tasks, and GitHub pushes and pull requests linked by those keys. Everything is stored so a later AI summary can read it. Video calls and AI summaries come next.

Phase 1: core workspace

Complete

Accounts, teams and roles, invitations, projects, tasks with a Kanban board, comments, task history, notifications, a dashboard, and a new landing page.

Phase 2: real-time teamwork

Complete

The live connection, live updates, presence, live comments, project discussions, meeting records with action items that become tasks, and one stored project context for future summaries.

Phase 3: GitHub

Complete

Every task has a short key. Connecting a GitHub App records pushes and pull requests, links them when the key is in a branch, commit, or PR, and shows that code on the task and the project.

Timeline

The main milestones, taken from the project's commit history.

24 Aug 2026

First working version

The NestJS backend, the React frontend, and a first marketing site were set up.

15 Sep 2026

Backend hardening

Security work on the API, and the shell of the signed-in app (sidebar, routing, session handling).

1 Oct 2026

Phase 1 feature build

Sign-in, workspaces for owners, admins, and members, invitations, onboarding, projects, tasks, assignment, comments, and role management.

7 Oct 2026

Polishing the collaboration flows

Clearer permissions on screen, better empty and error states, and steadier forms.

8 Oct 2026

Fixes from manual testing, and the new landing page

Exact task history, descriptive notifications, a redesigned dashboard, and correct due-date handling. Also a scroll-driven landing story, due dates on board cards, link-sharing previews, and a mobile menu.

10 Oct 2026

Phase 2 begins: live updates and presence

A secure live connection, screens that refresh on their own, a list of who is online, and an "also here" indicator on projects and tasks.

11 Oct 2026

Admin notifications

Workspace admins and the owner now hear about task and comment activity in every project, not just their own.

13 Oct 2026

Phase 2 complete: discussions and meetings

A live discussion on every project, meeting records with notes, decisions, and action items that turn into tasks, and one "project context" endpoint that gathers it all for future summaries.

20 Oct 2026

Task keys

Each workspace has a prefix and a counter. New tasks get the next number inside the same save, existing tasks were numbered in creation order, and keys show on boards, lists, the dashboard, and the task page.

21 Oct 2026

Phase 3 complete: GitHub

Workspace admins connect a GitHub App. Signed webhooks store repositories, commits, and pull requests, link them by task key, notify the assignee when a PR is reviewed or merged, and add that code history to the project context.

How the pieces fit together

The browser talks to one API server in two ways. Normal requests save and load data, and a live connection pushes "something changed" signals back. All data is stored in PostgreSQL.

Browser app

React screens. Data is cached with TanStack Query and refreshed when the server signals a change.

React 19 · Vite · Tailwind v4 · dnd-kit

REST + cookies
Socket.IO

NestJS API

Modules for auth, users, workspaces, projects, tasks, comments, messages, meetings, GitHub, project context, activity, notifications, the dashboard, and realtime. Every request is checked against the user's workspace role.

JWT cookies · argon2 · Rate limits · Swagger docs

Prisma ORM

PostgreSQL

Every team record carries its workspace id, which keeps each team's data separate.

24 tables | Transactions | Row locks

Backend

The server holds the rules. The frontend hides buttons people can't use, but the backend always makes the final permission check.

Features

Accounts and sessions

- Register, sign in, sign out, and refresh the session
- Passwords hashed with argon2
- Short-lived access cookie (15 minutes) and a rotating refresh token, stored hashed
- Sign-in limited to 8 attempts per minute

Workspaces and roles

- Owner, admin, and member roles
- Invite links that expire
- Only the owner can invite admins
- Owners and admins change roles and remove members

Projects

- Create, edit, archive, restore, and delete
- Status, priority, and start and due dates (start must come before due)
- Project members
- Only the creator or an admin can manage a project

Tasks

- Four columns: To do, In progress, In review, Done
- Priority, assignee, due date, and description
- Safe reordering with a row lock, so two people moving cards never collide
- Members can update only their own tasks, and only the status and description

History and notifications

- Every change is recorded in plain words, for example "changed due date from 3 Oct to 10 Oct"
- A separate audit trail for sensitive actions
- Notifications with read and read-all
- Saves that change nothing are not recorded

Dashboard

- Real counts: open, overdue, due soon, and stale tasks
- My tasks, recent projects, and recent activity
- Due dates are compared in UTC, so time zones don't shift them

Project discussion

Phase 2

- Any workspace member can read and post in a project
- Authors edit their own messages; authors and admins can delete
- Loads 50 at a time, with older pages on request
- New messages appear live for people with the project open, notify the project's followers, and stay out of the activity feed

Meeting records

Phase 2

- Title, start and end time, participants, notes, decisions, and action items
- Invited people get a notification and can edit the record
- Only the organizer, the project creator, or an admin can delete it
- Note edits refresh open screens without filling the feed

Action items become tasks

Phase 2

- Uses the normal task rules, so members still can't set an assignee or due date
- Each item can become a task only once, even if two people click at the same time
- The task keeps a link back to its meeting

Task keys

Phase 3

- A short key per task, such as `GS-42`, from the workspace prefix and a number
- The next number is taken inside the same save as the task, so two creates cannot share a number
- Admins can change the prefix in workspace settings

GitHub connection

Phase 3

- Owners and admins connect the GitHub App from workspace settings
- The redirect is tied to that workspace with a short-lived signed token
- Each repository can be assigned to a project, which puts its code on that project's Code tab

Stored project context

Phase 2

One endpoint, `GET /projects/:id/context`, returns a project's tasks (including each task's number), its latest 200 discussion messages, every meeting with its participants, notes, decisions, and action items, the latest 100 activity entries, and now the latest 100 pull requests and 100 commits linked to the project. Lists read oldest first, like a timeline, and each task says which meeting it came from. This is the input the AI summaries will use.

GitHub webhooks and linking

Phase 3

GitHub signs every webhook with HMAC over the raw body. Each delivery id is stored, so a retry does not create the same commit twice. A push, pull request, or review is linked to a task when the workspace prefix and number appear in the branch name, commit message, or pull request title or description. Merging or reviewing a linked PR notifies the assignee. Merging does not move the task; if it is not already Done, the task page offers a "Mark as done" prompt.

Data model

AREA	TABLES	WHAT THEY STORE
People	`User`, `RefreshToken`	Accounts, and sign-in sessions that can be revoked
Teams	`Workspace`, `WorkspaceMember`, `WorkspaceInvite`	Teams, each person's role, and pending invitations
Work	`Project`, `ProjectMember`, `Task`, `Comment`	The work itself and the discussion on it
Conversations and meetings	`Message`, `Meeting`, `MeetingParticipant`, `MeetingDecision`, `MeetingActionItem`	Project discussions, and what each meeting decided and who follows up. One message shape can later serve task and workspace chat too.
Context	`ActivityEvent`, `Notification`, `AuditEvent`	What happened, who needs to know, and a record for accountability. Activity from GitHub has no Grasyn actor; the GitHub login is stored in the event details.
GitHub	`GitHubInstallation`, `Repository`, `Commit`, `CommitTask`, `PullRequest`, `PullRequestTask`, `GitHubDelivery`	The connected GitHub App, the repos it can see, commits and pull requests, which tasks they mention, and delivery ids so retries count once

API endpoints (66)

MODULE	ENDPOINTS
Auth and users	register · login · refresh · logout · me · update profile

Workspaces	create · list · get · update (name or task prefix) · members · invite · view invite · accept invite · change role · remove member
Projects	create · list · get · update · archive · delete · add member · remove member
Tasks	create · list per project · list per workspace (with filters) · get · update · move on board
Comments	list · add
Messages	list with paging · post · edit · delete · mark the discussion read
Meetings	list per project · create · get · update · delete · add or remove a participant · add or remove a decision · add or remove an action item · turn an action item into a task
Project context	everything about one project, in one response
Activity	for a workspace · for a project · for a task
Notifications	list with unread count · mark one read · mark all read
GitHub	status · connect · disconnect · assign a repository to a project · code on a task · code on a project · setup callback from GitHub · signed webhook
Dashboard and health	dashboard summary · health check

Realtime layer (Phase 2)

Secure connection

The live connection uses the same sign-in cookie as the API. A missing or invalid session is disconnected right away. Joining a workspace, project, or task "room" runs the same membership checks as the API.

Only real changes are sent

Signals are queued while a save is running and sent only after it succeeds. If the database rejects the save, nothing is sent, so other screens never refresh to data that doesn't exist.

Presence

The server tracks which project or task each open tab is showing. A person with several tabs counts once, and stays online until their last tab closes.

Notifications for the right people

The assignee, the project creator, and now every workspace admin and the owner. Each person gets one notification per change, and whoever made the change gets none.

Someone saves a change → Permission check → Database save →

History and notifications written → Live signal sent → Teammates' screens refresh

Who gets notified

The current rules, including the fix for admins. Whoever made the change is never notified about their own action.

EVENT	ASSIGNEE	PROJECT CREATOR	WORKSPACE OWNER AND ADMINS	PERSON WHO MADE THE CHANGE
New task created	Yes (assigned)	Yes	Yes	No
Task changed (status, priority, date, details)	Yes	Yes	Yes	No
Task reassigned	Yes (new assignee)	Yes	Yes	No
New comment	Yes	Yes	Yes	No
Linked pull request reviewed or merged	Yes	No	No	— (GitHub user)

If one person holds several of these roles, for example an admin who is also the assignee, they still get only one notification. When someone is added to a meeting, they are notified too, unless they added themselves. A new discussion message notifies the project creator, the project's members, the workspace owner and admins, anyone with a task in the project, and anyone who has posted in that discussion. Each person keeps

at most one unread discussion notification per project, which updates with the latest message, and it clears when they open the Discussion tab.

Frontend

One React app with two sides: a public landing story, and the signed-in workspace.

Screens

SCREEN	WHAT PEOPLE CAN DO THERE
Landing page	A scroll-driven story: a dark animated hero, a "lights on" transition, then chapters on the problem, its cost, the idea, how it works, features, and pricing. Includes a mobile menu and link-sharing previews.
Sign in and register	Email and password, a password strength meter, and smooth page transitions. Google and GitHub buttons are shown but marked as not yet available.
Onboarding	Create your first workspace, or accept an invitation.
Dashboard	Overview counts, my open and overdue tasks, due soon, stale work, recent projects, and team activity.
Projects and project detail	Browse, create, edit, archive, and delete projects. The project page has four tabs: Tasks, a live Discussion, Meetings, and Code (pull requests and commits from assigned repositories, plus anything that names a task in the project).
Meeting page	The meeting's time and participants, shared notes, decisions, and action items. Each action item has a "Create task" button, and once converted it links to the task and shows its status.
Kanban board	Drag cards between columns. Cards show the task key, the due date, and an overdue warning. Each person can move only the cards they are allowed to.
Task detail	The task key (with copy), edit fields according to your role, a Code section of linked commits and pull requests, a "Mark as done" prompt after a merge, the full change history, and comments.
My tasks, Activity, Notifications	A personal task list, a workspace-wide history feed, and notifications. The unread badge clears when the notifications page opens.

Settings and profile	Invite people, change roles, remove members, change the task-key prefix, connect or disconnect GitHub, assign repositories to projects, and edit your profile.

What makes it feel good

Live without refreshing

When a teammate moves a task or comments, the board, task page, dashboard, and notification badge update by themselves.

See your team

The sidebar shows who is online now. Projects, boards, and tasks show the faces of anyone else on the same page.

Clear permissions

Each screen says what you are allowed to do, for example "Execution access", and hides the actions you can't take.

Steady layout

The sidebar stays in place and only the content area scrolls. Every screen has loading, empty, and error states.

Motion with care

The landing animations use GSAP and Lenis smooth scrolling. People who turn off motion in their system settings get a calm, static version.

Fast to load

The animation libraries load only on the landing page (about 68 KB compressed), so the workspace app stays light.

Talk where the work is

Each project has its own discussion. Enter sends, Shift+Enter adds a line, and new messages appear for everyone on the page.

Meetings that lead to work

Decisions and follow-ups are written down during the meeting, and a follow-up becomes a board task in one click.

Edits that don't collide

Unsaved meeting notes are kept while teammates make changes, and the app clearly marks what is not saved yet.

Code next to the task

Put the task key in a branch, commit, or pull request, and that work appears on the task and on the project's Code tab as it happens.

Quality checks

Automated tests: 107 passing

- Permissions for every role: who can create, edit, move, and manage
- Sign-in, sessions, and configuration checks, including a complete or half-filled GitHub App setup
- Notification rules for tasks, comments, and meeting invitations
- Plain-language task history, including GitHub pushes, reviews, and merges
- Live connection: rejects bad sessions, checks room access, and never sends signals for failed saves
- Presence: merges tabs, handles leaving and switching workspaces
- Discussions: paging, author-only edits, admin deletes, and who gets notified
- Meetings: who can edit, times that make sense, and an action item can't become two tasks
- GitHub helpers: app tokens verify with RS256, and webhook signatures reject a changed body or the wrong secret
- Task keys: prefix from a workspace name, unique numbers from text, and no match inside a longer word
- 5 end-to-end tests against a real PostgreSQL database, including a full discussion, meeting, project-context run, and a GitHub connect, signed webhook, link-by-key, and duplicate-delivery run

Other checks

- Code style checks (lint) and production builds pass for both apps
- The landing page was checked in a browser on desktop, on mobile, and with motion turned off
- Manual testing with several accounts: tasks, comments, online status, and "also here" all confirmed working
- Discussions and meetings are covered by the automated tests above and are ready for the same hands-on testing

What comes next

Finished work, and what follows it, in the planned order.

STEP	STATUS	NOTES
1. Secure live connection and workspace rooms	Done	Cookie sign-in, with room access checked like the API

2. Live task, comment, and notification updates	Done	Signals are sent only after a successful save
3. Presence: who is online and who is here	Done	Kept in server memory; one server for now
4. Live comments	Done	New comments appear on open task pages right away
5. Project discussion threads	Done	One message model, ready for task and workspace chat later
6. Meeting records and action items	Done	Notes and decisions, with action items that become real tasks
6b. Stored project context	Done	One endpoint that gathers tasks, discussion, meetings, and now code
7. Task keys	Done	Workspace prefix plus a number, shown throughout the app
8. GitHub App, webhooks, and code views	Done	Connect from workspace settings; code links by task key
9. Video calls	Next	Using LiveKit, attached to a meeting record
10. AI summaries	Planned	Reads the stored project context, including code history

Known limits to keep in mind

Presence covers one server. It is kept in memory. If the app later runs on several servers, it will need a shared store such as Redis.

Google and GitHub sign-in are not built yet. The buttons are there but clearly marked as unavailable.

The link-preview image needs the real domain. Once the site is live, the preview tag must use the full address.

Notifications are in-app only. There are no email or push notifications yet.

Discussions are plain text. There are no @mentions, files, or reactions yet.

Meeting times use each person's browser time zone. There is no calendar sync or reminder yet.

Changing a task prefix stops old keys from linking. Code that still mentions `OLD-12` will not match the new prefix.

Disconnecting GitHub in Grasyn does not uninstall the app on GitHub. Webhooks for that install are ignored until someone connects it again, or uninstalls it on GitHub.

Local GitHub webhooks need a relay. GitHub cannot reach localhost, so a tool such as smee.io forwards signed events to the API.

This report is updated at the end of every phase, from the project source and commit history. Current edition: 21 October 2026.